

UNIDAD 6:

TENSORFLOW

CURSO

IA

```
01 import openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10
11     respuesta = modelo.invoke(
12         prompt
13     )
14     return respuesta.content
15
16 if __name__ == "__main__":
17     prompt = "Explicame qué
18     es la inteligencia artificial"
19     print(generar_respuesta(prompt))
```



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com

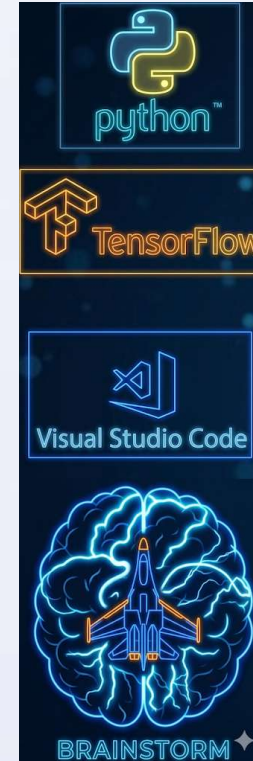


Tel: +54 351 3733383



Entorno de trabajo:

ENTORNO DE TRABAJO	
Lenguaje de Programación:	Python
Librería de desarrollo IA:	TensorFlow
Entorno de Desarrollo (IDE):	VSCode
Herramienta Didáctica:	AIR Battle
Herramienta Didáctica:	Lienzo MNIST





Python y TensorFlow: Verificar Funcionamiento Entorno.

Pasos desde el CMD de Windows:

1. Verificar que Python esté instalado

```
python --version
```

Debería devolver algo como: Python 3.10.12

Si no lo reconoce, agregá Python al PATH o usá py en lugar de python.

2. Verificar que TensorFlow esté instalado

```
pip show tensorflow
```

Te mostrará información como: Name: tensorflow Version: 2.15.0

Si dice "Package(s) not found", entonces TensorFlow no está instalado en ese entorno.

3. Verificar desde CMD que TensorFlow responde correctamente

Sin necesidad de importar nada, podés pedirle a Python que ejecute un comando inline que importe TensorFlow y lo use:

```
python -c "import tensorflow as tf; print(tf.__version__)"
```

Si funciona, te mostrará la versión instalada de TensorFlow (por ejemplo, 2.15.0).

4. (Opcional) Verificar que TensorFlow detecta una GPU

```
python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

Devolverá una lista de dispositivos GPU si está bien configurado, o una lista vacía si solo se usa CPU.

```
Administrator: Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\WINDOWS\system32> python --version
>>
Python 3.8.2
PS C:\WINDOWS\system32> pip show tensorflow
Name: tensorflow
Version: 2.13.0
Summary: TensorFlow is an open source machine learning framework for everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: c:\users\pc-master\appdata\local\programs\python\python38\lib\site-packages
Requires: tensorflow-intel
Required-by:
PS C:\WINDOWS\system32> python -c "import tensorflow as tf; print(tf.__version__)"
>>
2.13.0
PS C:\WINDOWS\system32>
```



En caso de errores frecuentes:

Mensaje de error	Posible causa	Solución rápida
ModuleNotFoundError: No module named 'tensorflow'	TensorFlow no está instalado	<code>pip install tensorflow</code>
msvcp140.dll is missing	Falta Visual C++ Redistributable	Instalá desde el sitio oficial de Microsoft
ImportError: DLL load failed	Conflicto con la arquitectura (32/64 bits)	Reinstalá Python y TensorFlow con la misma arquitectura



VSCODE, Python y TensorFlow: "Hola Mundo"

Pasos desde VS CODE:

1. Abrir VSCODE
2. File >> New text file
3. En la pantalla de trabajo >> "Select a Language"
4. "Python"

Primer Programa de verificación de entorno en VSCODE:

```
print("-----")
print("Hola Mundo!!")
print("Importando Libreria TF...")

import tensorflow as tf

print("Version tensorflow: ")
print(tf.__version__)
print("-----")
```

The screenshot shows the Visual Studio Code interface. The editor has two tabs: 'Untitled-1' and 'Untitled-3'. The code in 'Untitled-1' is the same as shown in the previous block. The right sidebar shows the 'Select a Language' menu with 'Python' selected. The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\PC-MASTER> python -u "C:\Users\PC-MAS~1\AppData\Local\Temp\tempCodeRunnerFile.python"
-----
Importando Libreria TF...
Version tensorflow:
2.13.0
-----
PS C:\Users\PC-MASTER> python -u "C:\Users\PC-MAS~1\AppData\Local\Temp\tempCodeRunnerFile.python"
-----
Hola Mundo!!
Importando Libreria TF...
Version tensorflow:
2.13.0
-----
PS C:\Users\PC-MASTER>
```

The status bar at the bottom indicates: Ln 12, Col 1, Spaces: 4, UTF-8, CRLF, Python, 3.8.2 64-bit.



TENSORFLOW [HISTORIA]

La historia de TensorFlow, una de las bibliotecas de inteligencia artificial más populares del mundo, está íntimamente ligada al crecimiento del aprendizaje profundo y al papel de Google en su desarrollo y expansión.

Orígenes: DistBelief (2011–2015): Antes de TensorFlow, Google desarrolló internamente un sistema llamado DistBelief, una infraestructura de aprendizaje profundo diseñada para sus propias necesidades.



Nacimiento de TensorFlow (2015):

Para superar esas limitaciones, el equipo de Google Brain —liderado por Jeff Dean, Rajat Monga y otros— comenzó a trabajar en una nueva biblioteca desde cero. Así nació TensorFlow, con los siguientes objetivos clave:

- Código abierto desde el principio.
- Flexible, modular y extensible.
- Capaz de ejecutarse en CPU, GPU y dispositivos móviles.
- Basado en gráficos computacionales (dataflow graphs).

Expansión global y adopción (2016–2018):

Desde su lanzamiento, TensorFlow ganó tracción rápidamente por varias razones:

- Soporte oficial de Google.
- Documentación extensa y comunidad activa.
- Integración con Keras, una API de alto nivel.

TensorFlow 2.x (2019–presente)

Con el tiempo, los usuarios comenzaron a criticar la complejidad de TensorFlow 1.x: muchos consideraban que era difícil de usar, especialmente para principiantes. En respuesta, en 2019 Google lanzó TensorFlow 2.0, una versión completamente rediseñada con:

- Eager Execution (evaluación inmediata de operaciones).
- Keras como API oficial de alto nivel.
- Mejor integración con NumPy y Python.
- Simplificación de funciones y eliminación de redundancias.
- Esta nueva versión hizo que TensorFlow fuera más intuitiva, especialmente para investigadores y educadores.



TENSORFLOW [ECOSISTEMA]

TensorFlow es un ecosistema modular de aprendizaje automático desarrollado por Google, que abarca desde la investigación y entrenamiento hasta el despliegue en producción.

Incluye herramientas para visión por computadora, procesamiento de lenguaje natural, móviles, navegadores y la nube.

Se integra fácilmente con bibliotecas de datos y aprendizaje clásico como NumPy, Pandas y Scikit-learn.



Componente	Función principal	Integración destacada
TensorFlow Core	Núcleo de ejecución y gráficos computacionales	NumPy, CUDA, XLA
Keras	API de alto nivel para modelos	Scikit-learn, TensorBoard
TensorBoard	Visualización de métricas y arquitectura	Compatible con Keras y Core
tf.data / TFDS	Manejo eficiente de datos	Pandas, TF Datasets
TF Lite	Despliegue en móviles y microcontroladores	Android, Arduino, Raspberry Pi
TF.js	Ejecución en navegador con JavaScript	WebGL, Node.js
TFX	Pipeline de producción y MLOps	Apache Airflow, Kubeflow
TensorFlow Hub	Repositorio de modelos preentrenados	Reutilizable con Keras y Core
TensorFlow Serving	Despliegue de modelos en servidores	REST/gRPC, Docker
ONNX / tf2onnx	Interoperabilidad entre frameworks	PyTorch, Caffe2, MXNet
OpenCV	Visión por computadora	Preprocesamiento de imágenes
Google Cloud AI	Entrenamiento y despliegue en la nube	AutoML, BigQuery ML



PROCESO DE ENTRENAMIENTO DE UNA RED NEURONAL

Para desarrollar y utilizar una red neuronal profunda (DNN), es fundamental seguir una secuencia estructurada de etapas que abarca desde su definición inicial hasta su uso en producción.

A continuación, se presenta un resumen en 8 pasos que describe el flujo general de trabajo en el desarrollo de una DNN, permitiendo comprender el proceso completo antes de profundizar en los detalles técnicos de cada fase.



Paso	Etapas	Descripción general
1	Definir el modelo	Se construye la arquitectura de la red: cuántas capas tiene, qué tipo de capas son, cuántas neuronas hay en cada una, y cómo se conectan.
2	Compilar el modelo	Se elige el optimizador, la función de pérdida (loss) y las métricas que se van a evaluar durante el entrenamiento.
3	Preparar los datos	Se cargan y preprocesan los datos para alimentar al modelo, dividiéndolos normalmente en entrenamiento, validación y prueba.
4	Entrenar el modelo	El modelo ajusta sus pesos internamente al ver los datos de entrenamiento durante varias iteraciones (épocas), minimizando el error.
5	Evaluar el modelo	Se prueba el modelo con datos no vistos (validación o test) para medir su rendimiento real.
6	Guardar el modelo	Se almacena el modelo entrenado en disco para poder reutilizarlo sin tener que volver a entrenarlo.
7	Ajustar o mejorar	Se puede volver a entrenar, modificar la arquitectura, cambiar hiperparámetros, o aplicar técnicas como regularización.
8	Cargar y usar el modelo	Se vuelve a cargar el modelo guardado para hacer predicciones (inferencia) en nuevos datos.



INTRODUCCIÓN AL DESAFIO.

Vamos a construir un modelo de red neuronal que pueda **reconocer dígitos escritos a mano**, del 0 al 9, a partir de imágenes en escala de grises. Este es un problema de **clasificación multiclase**, ya que el modelo debe predecir a cuál de las 10 clases posibles (0, 1, ..., 9) pertenece cada imagen.

dataset MNIST

MNIST (Modified National Institute of Standards and Technology) es un conjunto de datos ampliamente utilizado en el campo del aprendizaje automático.

Contiene:

- 60,000 imágenes para entrenamiento
- 10,000 imágenes para prueba
- Cada imagen tiene un tamaño de 28x28 píxeles
- Las imágenes están en escala de grises
- Cada imagen contiene un solo dígito escrito a mano (0-9)
- Es ideal para probar y comparar arquitecturas de redes neuronales.

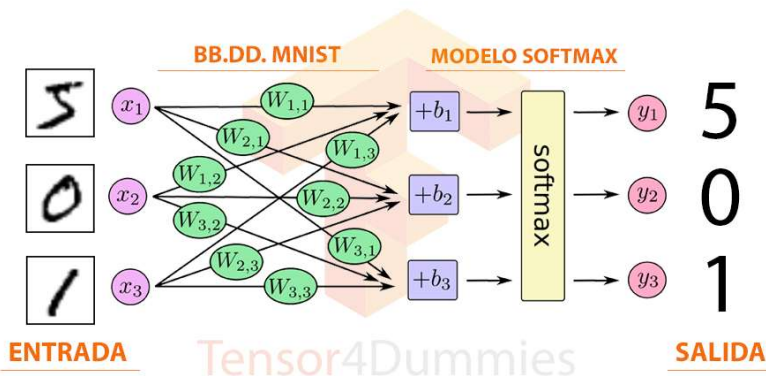
¿Dónde está ubicada la librería MNIST?

En TensorFlow, MNIST está disponible directamente a través de la API de Keras. No es necesario descargar nada manualmente. Puede accederse mediante el módulo:

Este módulo contiene una función `load_data()` que carga y divide automáticamente el dataset en:

- Datos de entrenamiento: (x_train, y_train)
- Datos de prueba: (x_test, y_test)

Esto hace que comenzar con MNIST en TensorFlow sea rápido y sencillo, ideal para demostrar todas las etapas de creación y entrenamiento de una red neuronal.





Paso 1: Definir el modelo

Definir un modelo significa construir su **arquitectura**, es decir, decidir:

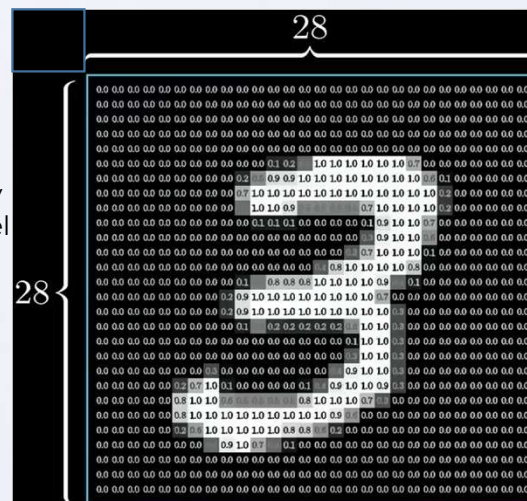
- Cuántas **capas** tendrá la red
- Qué **tipo de capas** se usarán (densas, convolucionales, etc.)
- Cuántas **neuronas** habrá en cada capa
- Qué **funciones de activación** se aplicarán
- Cómo se conectan las capas entre sí

Dado que vamos a trabajar con imágenes de 28x28 píxeles y resolver un problema de clasificación en 10 clases (dígitos del 0 al 9), usaremos una red neuronal **densa (fully connected)** simple para empezar.

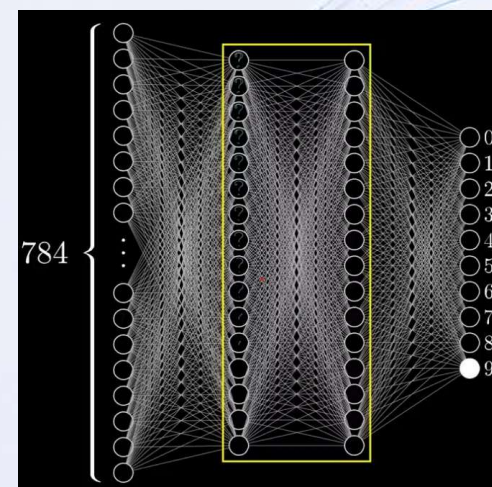
Arquitectura propuesta

- **Entrada:** imagen de 28x28 píxeles, que convertiremos a un vector de 784 valores (aplanamiento).
- **Primera capa oculta:** 50 neuronas con función de activación ReLU.
- **Segunda capa oculta:** 50 neuronas con función de activación ReLU.
- **Capa de salida:** 10 neuronas con función de activación softmax (para obtener probabilidades en las 10 clases).

ENTRADA: 28x28=784



MODELO PROPUESTO





Paso 1: Definir el modelo

Explicación de cada componente

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Aplanamiento (Flatten): transforma la imagen 2D en un vector 1D para que pueda ser procesado por las capas densas.

Capas ocultas (Dense): cada una con 50 neuronas y activación ReLU, que aporta la capacidad de aprender representaciones no lineales útiles para clasificar los dígitos.

Capa de salida (Dense): 10 neuronas, una por cada dígito posible, con activación softmax para obtener una distribución de probabilidad sobre las clases.

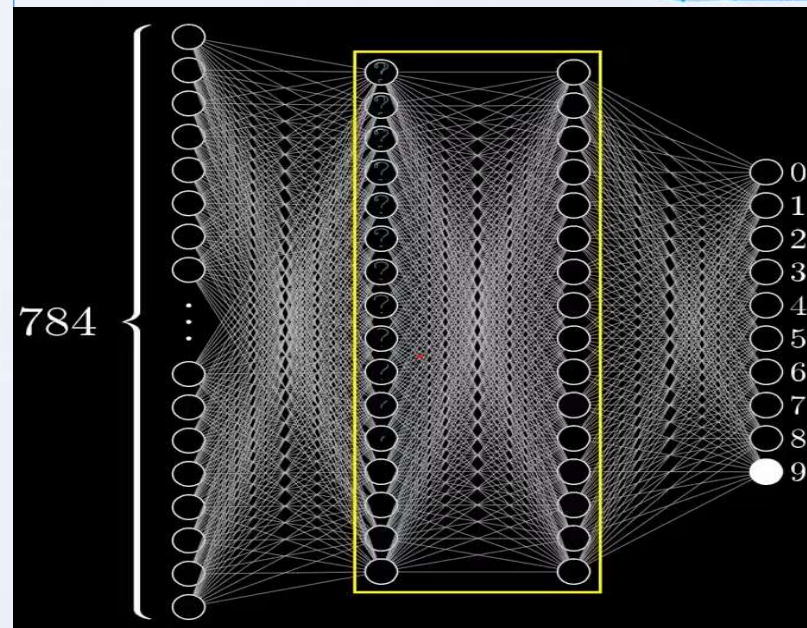
Aquí se presenta el código:

```
import tensorflow as tf
from keras import Sequential
from keras.layers import Input, Flatten, Dense, Dropout, BatchNormalization, Activation
from keras.initializers import HeNormal, GlorotUniform

model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(10, activation="softmax", kernel_initializer=GlorotUniform())
])
print("=====")
print("===== ESTRUCTURA DEL MODELO =====")
print("=====")
model.summary()
```



MODELO PROPUESTO



```
# PARA OCULTAR WARNINGS INICIALES
import os
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
os.environ["ABSL_LOG_LEVEL"] = "3"

import tensorflow as tf
```





Paso 1: Definir el modelo: [TIPOS DE CAPA]

Tabla resumen de los tipos de capas más comunes en Keras/TensorFlow, con su descripción y uso principal.

Esta tabla se centra en las capas más utilizadas al comenzar con redes neuronales.

Etapas	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(10, activation="softmax", kernel_initializer=GlorotUniform())
])
```



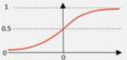
Nombre de la capa	Clase en Keras	Descripción breve	Uso principal
Dense	Dense	Capa totalmente conectada (fully connected): cada neurona recibe todas las entradas.	Redes neuronales clásicas, clasificación.
Flatten	Flatten	Aplana una entrada multidimensional a un vector 1D.	Transición de capas convolucionales a densas.
Input	Input	Define la forma de la entrada al modelo.	Modelos funcionales, redes complejas.
Conv2D	Conv2D	Capa convolucional 2D que aplica filtros a imágenes.	Visión por computadora, CNNs.
MaxPooling2D	MaxPooling2D	Reduce el tamaño espacial tomando el valor máximo en regiones.	Reducción de dimensiones en CNNs.
Dropout	Dropout	Apaga aleatoriamente neuronas durante el entrenamiento.	Prevención de sobreajuste (overfitting).
Batch Normalization	BatchNormalization	Normaliza las salidas de la capa anterior.	Estabiliza y acelera el entrenamiento.
Activation	Activation	Aplica una función de activación separada (ReLU, sigmoid, softmax, etc.).	Cuando se quiere separar la activación de la capa.
Reshape	Reshape	Cambia la forma de los datos sin alterar su contenido.	Preparación de datos para ciertas capas.
Embedding	Embedding	Convierte enteros en vectores densos.	Procesamiento de texto, NLP.
LSTM	LSTM	Capa de red neuronal recurrente con memoria.	Secuencias de texto o series temporales.
GRU	GRU	Variante más eficiente de LSTM.	Procesamiento secuencial.
Global Average Pooling2D	GlobalAveragePooling2D	Reduce cada mapa de características a un valor promedio.	CNNs antes de la capa de salida.



Paso 1: Definir el modelo: [FUNCIONES DE ACTIVACION]

Tabla con las funciones de activación más comunes

utilizadas en redes neuronales, con su descripción, fórmula y usos típicos.

Name	Formula	Derivative	Graph	Range
sigmoid (logistic function)	$\sigma(a) = \frac{1}{1+e^{-a}}$	$\frac{\partial \sigma(a)}{\partial a} = \sigma(a)(1 - \sigma(a))$		(0,1)
TanH (hyperbolic tangent)	$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$	$\frac{\partial \tanh(a)}{\partial a} = \frac{4}{(e^a + e^{-a})^2}$		(-1,1)
ReLU (rectified linear unit)	$\text{relu}(a) = \max(0, a)$	$\frac{\partial \text{relu}(a)}{\partial a} = \begin{cases} 0, & \text{if } a \leq 0 \\ 1, & \text{if } a > 0 \end{cases}$		(0,∞)
softmax	$\sigma_i(a) = \frac{e^{a_i}}{\sum_j e^{a_j}}$	$\frac{\partial \sigma_i(a)}{\partial a_j} = \sigma_i(a) (\delta_{ij} - \sigma_j(a))$ Where δ_{ij} is 1 if $i=j$, 0 otherwise	different every time	(0,1)

```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(10, activation="softmax", kernel_initializer=GlorotUniform())
])
```



Nombre	Fórmula (simplificada)	Rango de salida	Características principales	Usos comunes
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	No lineal, sencilla, rápida de calcular. Evita saturación positiva.	Capas ocultas en la mayoría de redes
Sigmoid	$f(x) = 1 / (1 + e^{-x})$	(0, 1)	Suave, salida probabilística. Se satura con valores extremos.	Clasificación binaria, salida de redes
Tanh	$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	(-1, 1)	Similar a sigmoid pero centrada en 0. Mejora el flujo de gradientes.	Capas ocultas (menos usada que ReLU)
Softmax	$f(x_i) = e^{x_i} / \sum e^{x_j}$	(0, 1), suma = 1	Vector de probabilidades. Útil para clasificación multiclase.	Capa de salida en clasificación multiclase
Leaky ReLU	$f(x) = x$ if $x > 0$ else αx ($\alpha \approx 0.01$)	$(-\infty, \infty)$	Variante de ReLU que permite pequeños gradientes cuando $x < 0$.	Capas ocultas, evita neuronas muertas
ELU	$f(x) = x$ if $x > 0$ else $\alpha(e^x - 1)$	$(-\alpha, \infty)$	Similar a Leaky ReLU pero más suave. Mejora estabilidad.	Capas ocultas en redes profundas
Swish	$f(x) = x * \text{sigmoid}(x)$	$(\sim -0.3, \infty)$	No lineal, suave, mejora el rendimiento en algunos casos.	Deep Learning avanzado (Google Research)
Linear	$f(x) = x$	$(-\infty, \infty)$	No transforma la entrada.	Salida en regresión



Paso 1: Definir el modelo [INICIADORES RANDOM]

Tabla resumen con los **kernel_initializer** más comunes de Keras/TensorFlow:

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(50, activation="relu", kernel_initializer=HeNormal()),
    Dense(10, activation="softmax", kernel_initializer=GlorotUniform())
])
```



Inicializador	Cuándo usarlo	Ideal para activaciones	Descripción breve
HeNormal()	Redes profundas con ReLU o variantes (leaky, elu)	ReLU, LeakyReLU, ELU	Basado en varianza (fan_in), mejora convergencia y evita gradientes muertos.
HeUniform()	Similar a HeNormal, pero con distribución uniforme	ReLU, LeakyReLU	Varianza escalada pero valores más uniformes.
GlorotNormal()	Arquitecturas no muy profundas / balanceadas	tanh, softmax	Inicialización Xavier (media 0, varianza controlada).
GlorotUniform() (default)	Caso general, redes no profundas	tanh, softmax, general	Bueno para la mayoría de los problemas básicos.
RandomNormal(mean, stddev)	Casos personalizados / control fino	Cualquiera (depende)	Distribución normal definida por usuario.
RandomUniform(minval, maxval)	Inicialización manual básica	Cualquiera (depende)	Distribución uniforme entre valores definidos.
Zeros()	Casos experimentales (no recomendado en general)	-	Pone todos los pesos en cero → NO se aprende nada (rompe simetría).
Ones()	Casos muy puntuales (no recomendado)	-	Similar a Zeros(), rompe la simetría.
VarianceScaling()	Inicialización genérica configurable	Cualquier activación	Base de He y Glorot, permite definir fan_in, fan_out, etc.
Orthogonal()	Modelos muy profundos o recurrentes (RNNs)	Recurrentes (LSTM, GRU)	Mantiene estabilidad de gradientes en redes profundas.



Paso 2: Compilar el modelo

Una vez definida la arquitectura del modelo, el siguiente paso es **compilarlo**, lo cual significa decirle a TensorFlow **cómo va a aprender**.

En este paso se especifican tres cosas clave:

1. Función de pérdida (loss)

Es la **medida del error** entre la predicción del modelo y el valor real. El modelo intenta **minimizar esta función** durante el entrenamiento.

Para el caso de clasificación multiclase como MNIST (con etiquetas enteras del 0 al 9), la función adecuada es:

- **sparse_categorical_crossentropy**

2. Optimizador (optimizer)

Es el algoritmo que ajusta los **pesos de la red** para reducir la función de pérdida. El más común y eficiente en la mayoría de los casos es:

- **adam (Adaptive Moment Estimation)**

3. Métricas (metrics)

Son las medidas que queremos ver durante el entrenamiento para evaluar el rendimiento. La más común en clasificación es:

- **accuracy (porcentaje de predicciones correctas)**

Explicación:

- **optimizer='adam'**: usa Adam para actualizar los pesos.
- **loss='sparse_categorical_crossentropy'**: adecuada para clasificación multiclase con etiquetas enteras.
- **metrics=['accuracy']**: muestra el porcentaje de aciertos en cada época.



```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
print("=====")
print("===== CONFIGURACIÓN DEL MODELO =====")
print("=====")
print(f"Compilado      : {model.compiled}")
print(f"Optimizador     : {model.optimizer.__class__.__name__}")
print(f"Loss           : {model.loss}")
print("=====")
print("===== OPTIMIZADOR =====")
print("=====")
for k, v in model.optimizer.get_config().items():
    print(f"{k:<30} {v}")

#print("=====")
#print("===== COMPILE CONFIG =====")
#print("=====")
#from pprint import pprint
#pprint(model.get_compile_config(), sort_dicts=False)
```



Paso 2: Compilar el modelo [TIPOS DE SALIDAS]

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Tabla con los tipos de *outputs* más comunes en redes neuronales, incluyendo:

- La forma del *output*
- La activación recomendada
- La función de pérdida correspondiente
- El caso de uso típico

Glosario:

N: cantidad de muestras

C: cantidad de clases

M: cantidad de salidas numéricas

El **formato one-hot** es una forma de representar **categorías** usando **vectores binarios** donde:

Clase original	Representación <i>one-hot</i>
0	[1, 0, 0, 0]
1	[0, 1, 0, 0]
2	[0, 0, 1, 0]
3	[0, 0, 0, 1]

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```



Tipo de salida	Forma del output	Activación	Función de pérdida	Caso de uso típico
Clasificación multiclase	(N, C)	softmax	sparse_categorical_crossentropy	MNIST, clasificación de imágenes con 1 etiqueta
Clasificación multiclase (one-hot)	(N, C)	softmax	categorical_crossentropy	Cuando las etiquetas están en formato one-hot
Clasificación binaria	(N, 1)	sigmoid	binary_crossentropy	Clasificación sí/no (spam, cáncer, etc.)
Clasificación multietiqueta	(N, C)	sigmoid	binary_crossentropy	Ej. etiquetas múltiples por imagen (ropa, clima)
Regresión (1 variable)	(N, 1)	linear (sin activación)	mean_squared_error o mean_absolute_error	Predicción de temperatura, precios
Regresión multivariable	(N, M)	linear	mean_squared_error, mae, etc.	Predicción de múltiples variables numéricas
Autoencoder (reconstrucción)	Igual que entrada	sigmoid, linear, relu	mse, binary_crossentropy	Reconstrucción de imágenes, compresión
Redes generativas (GANs)	Depende del diseño	sigmoid, tanh, linear	Especializadas (bce, hinge, etc.)	Generación de imágenes, audio, etc.
Red de clasificación ordinal	(N, C-1) o (N, C)	sigmoid, softmax	Varias (custom, ordinal loss)	Escalas ordenadas (ej. niveles de dolor)



Paso 2: Compilar el modelo: [FUNC. DE PERDIDA]

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Tabla funciones de pérdida más comunes.

Esta información te ayuda a elegir la función adecuada según el tipo de tarea (clasificación, regresión, etc.).

Tipo de error	Fórmula
Error cuadrático medio (MSE)	$\text{error} = \frac{1}{n} \sum_{i=1}^n (\text{etiqueta}_i - \text{predicción}_i)^2$
Error absoluto medio (MAE)	$\text{error} = \frac{1}{n} \sum_{i=1}^n \text{etiqueta}_i - \text{predicción}_i $
Entropía cruzada categórica	$\text{error} = - \sum_i \text{etiqueta}_i \log(\text{predicción}_i)$
Entropía cruzada binaria	$\text{error} = - [\text{etiqueta} \log(\text{predicción}) + (1 - \text{etiqueta}) \log(1 - \text{predicción})]$
Hinge loss	$\text{error} = \sum_i \max(0, 1 - \text{etiqueta}_i \cdot \text{predicción}_i)$
Kullback-Leibler (KL) Divergencia	$\text{error} = \sum_i \text{etiqueta}_i \log \left(\frac{\text{etiqueta}_i}{\text{predicción}_i} \right)$

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```



Nombre	Clase Keras	Tipo de problema	Descripción general
Mean Squared Error (MSE)	mean_squared_error	Regresión	Calcula el promedio del cuadrado de los errores. Penaliza fuertemente los errores grandes.
Mean Absolute Error (MAE)	mean_absolute_error	Regresión	Promedio del valor absoluto del error. Más robusto a outliers que MSE.
Huber Loss	huber	Regresión	Combinación entre MAE y MSE: sensible pero menos afectado por valores extremos.
Binary Crossentropy	binary_crossentropy	Clasificación binaria	Para etiquetas 0/1. Mide la diferencia entre la probabilidad predicha y la real.
Categorical Crossentropy	categorical_crossentropy	Clasificación multiclase (one-hot)	Se usa cuando las etiquetas están codificadas como vectores one-hot.
Sparse Categorical Crossentropy	sparse_categorical_crossentropy	Clasificación multiclase (enteros)	Igual que la anterior, pero las etiquetas son enteros (como 0, 1, ..., 9).
Kullback-Leibler Divergence	kullback_leibler_divergence	Clasificación probabilística	Mide la distancia entre dos distribuciones de probabilidad.
Cosine Similarity	cosine_similarity	Medición de similitud	Maximiza la similitud del ángulo entre vectores predicción y verdad.
Poisson	poisson	Conteo de eventos	Para problemas donde la salida representa tasas de ocurrencia.
Hinge	hinge	Clasificación binaria (SVM-like)	Utilizado en máquinas de soporte vectorial (SVM).



Paso 2: Compilar el modelo: [OPTIMIZADORES]

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Tabla con los optimizadores más comunes

Los optimizadores son algoritmos que ajustan los pesos del modelo durante el entrenamiento para minimizar la función de pérdida.

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```



Nombre	Clase en Keras	Descripción general	Ventajas principales	Casos de uso comunes
SGD	SGD	Gradiente descendente estocástico. Actualiza pesos con una tasa fija.	Simple, buen control, base para otros optimizadores.	Modelos simples, problemas clásicos.
Momentum	SGD(momentum=...)	Variante de SGD que acumula velocidad en la dirección del gradiente.	Acelera la convergencia, evita oscilaciones.	Clasificación, redes profundas.
Nesterov	SGD(nesterov=True)	Extiende Momentum evaluando el gradiente antes del paso.	Más estable y rápido que momentum clásico.	Modelos más complejos con oscilación.
RMSprop	RMSprop	Ajusta la tasa de aprendizaje por parámetro según su histórico de gradientes.	Maneja bien problemas no estacionarios.	RNNs, problemas secuenciales.
Adam	Adam	Combina Momentum y RMSprop. Ajusta tasas individualmente.	Muy popular, rápido, robusto y eficiente.	Uso general, visión por computadora, NLP, etc.
Adagrad	Adagrad	Tasa de aprendizaje adaptativa por parámetro, acumulando cuadrados de gradiente.	Útil cuando hay parámetros poco frecuentes.	NLP, datos dispersos (sparse).
Adadelata	Adadelata	Variante mejorada de Adagrad. No requiere tasa de aprendizaje manual.	Menos sensible a la configuración de hiperparámetros.	Tareas generales con datos ruidosos.
Adamax	Adamax	Variante de Adam basada en norma infinito.	Estable con grandes valores de parámetros.	Redes profundas con valores grandes.
Nadam	Nadam	Adam + Nesterov momentum.	Mejora la velocidad y estabilidad en algunos casos.	Deep learning avanzado.
FTRL	Ftrl	Optimización regularizada para problemas dispersos.	Eficiente para modelos muy grandes, en línea.	Publicidad online, regresión logística.



Paso 2: Compilar el modelo

[METRICAS]

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Las métricas se usan para monitorear el rendimiento del modelo durante y después del entrenamiento, **sin afectar el aprendizaje** (a diferencia de la función de pérdida).

Keras tiene un comportamiento inteligente llamado inferencia de métricas. Cuando compilas un modelo y pasas el string genérico **metrics=['accuracy']** (o simplemente 'acc'). Keras analiza la función de pérdida (loss) que elegiste y la forma de tus etiquetas (y_train) **para asignar automáticamente la clase de precisión exacta que necesitas.**

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```



Nombre	Clase en Keras	Tipo de problema	Descripción breve	Usos principales
Accuracy	accuracy o Accuracy()	Clasificación (binaria o multiclase)	Porcentaje de predicciones correctas.	Métrica general de rendimiento en clasificación.
Binary Accuracy	BinaryAccuracy()	Clasificación binaria	Exactitud para problemas con 2 clases (0 o 1).	Clasificación binaria, salida sigmoid.
Categorical Accuracy	CategoricalAccuracy()	Multiclase (one-hot)	Compara vectores one-hot con las salidas del modelo.	Clasificación multiclase con etiquetas codificadas.
Sparse Categorical Accuracy	SparseCategoricalAccuracy()	Multiclase (etiquetas enteras)	Compara etiquetas enteras con las predicciones softmax.	MNIST, clasificación multiclase clásica.
Precision	Precision()	Clasificación (binaria o multi)	Porcentaje de verdaderos positivos entre los predichos como positivos.	Tareas donde los falsos positivos son costosos.
Recall	Recall()	Clasificación (binaria o multi)	Porcentaje de verdaderos positivos entre los casos reales positivos.	Tareas donde los falsos negativos son costosos.
AUC (ROC)	AUC()	Clasificación binaria	Área bajo la curva ROC. Evalúa la calidad general de clasificación.	Detección de desequilibrios entre clases.
Mean Absolute Error	MeanAbsoluteError()	Regresión	Promedio de los errores absolutos.	Problemas de regresión (valores continuos).
Mean Squared Error	MeanSquaredError()	Regresión	Promedio de los errores al cuadrado. Penaliza errores grandes.	Regresión, especialmente con valores estables.
Root Mean Squared Error	RootMeanSquaredError()	Regresión	Raíz cuadrada del MSE.	Interpretación más intuitiva que MSE.
Cosine Similarity	CosineSimilarity()	Similitud (clasificación o embeddings)	Mide cuán alineados están los vectores de salida y objetivo.	NLP, modelos de recomendación, embeddings.



Paso 3: Preparar los datos [IMPORTAR DATASET MNIST]

El conjunto de datos **MNIST** proviene del *National Institute of Standards and Technology (NIST)* y fue adaptado por Yann LeCun para ser más accesible y útil en la investigación y enseñanza del aprendizaje automático

- Cada imagen sea de **28x28 píxeles**
- Los valores de los píxeles estén entre **0 y 255** (blanco y negro)

La división entre entrenamiento y prueba **no la hace tu código**, sino que ya está definida en el propio dataset original:

- **60,000 imágenes para entrenamiento**
- **10,000 imágenes para prueba**

En este paso vamos a:

1. **Cargar el dataset MNIST**
2. **Normalizar las imágenes** (escalarlas entre 0 y 1)
3. **Verificar las dimensiones**

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

```
from keras.datasets import mnist
# Cargar imágenes y etiquetas
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Convertir a float y escalar entre 0 y 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

```
print("\n=====")
print("===== MNIST DATASET =====")
print("=====")
print(f"Train Images      : {x_train.shape}")
print(f"Train Labels       : {y_train.shape}")
print(f"Test Images        : {x_test.shape}")
print(f"Test Labels        : {y_test.shape}")
print("\n----- IMAGE INFO -----")
print(f"Image Size         : {x_train.shape[1]} x {x_train.shape[2]}")
print(f"Image Dtype         : {x_train.dtype}")
print(f"Pixel Min           : {x_train.min():.3f}")
print(f"Pixel Max           : {x_train.max():.3f}")
print("\n----- CLASSES -----")
classes = np.unique(y_train)
print(f"Number of Classes : {len(classes)}")
print(f"Classes           : {classes}")
print("\n----- LABEL DISTRIBUTION -----")
for digit in classes:
    count = np.sum(y_train == digit)
    print(f"Digit {digit}: {count:5d} samples")
print("\n=====")
```



Paso 3: Preparar los datos [PANDAS: MANEJO DE DATASET]

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Tomar datos desde un CSV con la librería PANDAS:

PANDAS es una librería de Python muy popular para **análisis y manipulación de datos tabulares**, es decir, datos organizados en filas y columnas, como una planilla de Excel o un archivo CSV. Su estructura principal es el **DataFrame**, que es como una tabla en memoria: cada columna puede tener un tipo diferente de dato (números, texto, fechas, etc.).

¿Por qué se usa pandas?

- Permite **leer datos desde archivos** CSV, Excel, SQL, etc.
- Hace fácil **seleccionar, filtrar, modificar y combinar columnas**.
- Se integra perfectamente con librerías de machine learning como **TensorFlow**, **scikit-learn** y **NumPy**.

```
import pandas as pd
```

```
# Ruta al archivo CSV (puede ser local o URL)
DS_PATH = 'ruta/al/dataset.csv'
```

```
# Cargar el dataset, saltando la primera fila si no
# contiene los nombres de las columnas
df = pd.read_csv(DS_PATH, header=1)
```

```
# Seleccionar las columnas de entrada (features)
X_DATA = df[['ENTRADA01', 'ENTRADA02', 'ENTRADA03',
'ENTRADA04', 'ENTRADA05', 'ENTRADA06', 'ENTRADA07',
'ENTRADA08', 'ENTRADA09', 'ENTRADA10', 'ENTRADA11',
'ENTRADA12', 'ENTRADA13', 'ENTRADA14', 'ENTRADA15',
'ENTRADA16', 'ENTRADA17', 'ENTRADA18', 'ENTRADA19',
'ENTRADA20', 'ENTRADA21', 'ENTRADA22', 'ENTRADA23',
'ENTRADA24', 'ENTRADA25', 'ENTRADA26', 'ENTRADA27',
'ENTRADA28', 'ENTRADA29', 'ENTRADA30', 'ENTRADA31',
'ENTRADA32', 'ENTRADA33', 'ENTRADA34',
'ENTRADA35', 'ENTRADA36', 'ENTRADA37', 'ENTRADA38',
'ENTRADA39', 'ENTRADA40']]
```

```
# Seleccionar las columnas de salida (etiquetas)
Y_DATA = df[['SALIDA1', 'SALIDA2', 'SALIDA3', 'SALIDA4',
'SALIDA5', 'SALIDA6', 'SALIDA7', 'SALIDA8']]
```





Paso 3: Preparar los datos [PANDAS: MANEJO DE DATASET]

Tabla con las funciones más comunes de PANDAS para trabajar con CSV:

```
import pandas as pd

# Ruta al archivo CSV (puede ser local o URL)
DS_PATH = 'ruta/al/dataset.csv'

# Cargar el dataset, saltando la primera fila si no contiene los nombres de las columnas
df = pd.read_csv(DS_PATH, header=1)

# Seleccionar las columnas de entrada (features)
X_DATA = df[['ENTRADA01', 'ENTRADA02', 'ENTRADA03',
'ENTRADA04', 'ENTRADA05', 'ENTRADA06', 'ENTRADA07', 'ENTRADA08',
'ENTRADA09', 'ENTRADA10', 'ENTRADA11', 'ENTRADA12', 'ENTRADA13',
'ENTRADA14', 'ENTRADA15', 'ENTRADA16', 'ENTRADA17', 'ENTRADA18',
'ENTRADA19', 'ENTRADA20', 'ENTRADA21', 'ENTRADA22', 'ENTRADA23',
'ENTRADA24', 'ENTRADA25', 'ENTRADA26', 'ENTRADA27', 'ENTRADA28',
'ENTRADA29', 'ENTRADA30', 'ENTRADA31', 'ENTRADA32', 'ENTRADA33',
'ENTRADA34', 'ENTRADA35', 'ENTRADA36', 'ENTRADA37', 'ENTRADA38',
'ENTRADA39', 'ENTRADA40']]

# Seleccionar las columnas de salida (etiquetas)
Y_DATA = df[['SALIDA1', 'SALIDA2', 'SALIDA3', 'SALIDA4',
'SALIDA5', 'SALIDA6', 'SALIDA7', 'SALIDA8']]
```



Función / Método	Descripción	Ejemplo / Opciones más comunes
pd.read_csv('archivo.csv')	Carga un archivo CSV como un DataFrame.	Por defecto, toma la primera fila como nombres de columnas.
pd.read_csv('archivo.csv', header=0)	Igual que arriba, explícitamente indica que la fila 0 es el encabezado.	Equivalente a header='infer'.
pd.read_csv('archivo.csv', header=1)	Usa la segunda fila (índice 1) como nombres de columnas.	Útil si la primera fila contiene metadatos o títulos.
pd.read_csv('archivo.csv', header=None)	No usa encabezado, asigna nombres automáticos a las columnas (0, 1, 2...).	Se puede combinar con names=[...] para nombrar columnas manualmente.
df.head(n)	Muestra las primeras n filas del DataFrame.	Por defecto, df.head() muestra las primeras 5 filas.
df.tail(n)	Muestra las últimas n filas.	Útil para ver el final del archivo.
df.columns	Lista los nombres de las columnas.	Ideal para verificar qué columnas tiene el CSV.
df[['col1', 'col2']]	Selecciona columnas específicas.	Se usa para extraer entradas (X) y salidas (Y).
df.iloc[filas, columnas]	Selecciona por índice (posición numérica).	Ejemplo: df.iloc[0:5, 0:3] muestra las primeras 5 filas y 3 columnas.
df.describe()	Muestra estadísticas básicas de columnas numéricas.	Media, desviación estándar, min, max, percentiles.
df.isnull().sum()	Muestra cuántos valores faltan por columna.	Útil para limpiar datos.
df.to_numpy()	Convierte el DataFrame en un array NumPy.	Útil para entrenar modelos con TensorFlow o scikit-learn.



Paso 4: Entrenar el modelo

En este paso usamos el método `fit()` de Keras para entrenar el modelo que definimos previamente con los datos ya preparados (`x_train, y_train`).

Método `.fit()`

- Este método hace que el modelo:
- **Recorra los datos varias veces (épocas).**
- Ajuste los **pesos internos** minimizando la función de pérdida.
- Muestre cómo mejora su rendimiento (métrica: accuracy).

Parámetro	Descripción
x y y	Datos de entrada (<code>x_train</code>) y etiquetas (<code>y_train</code>).
epochs	Número de veces que el modelo recorrerá todos los datos.
batch_size	Cantidad de muestras que se usan por paso de entrenamiento.
validation_split	Porcentaje del conjunto de entrenamiento reservado para validación.
verbose	Nivel de detalle en la consola (0, 1 o 2).

history: Almacena el registro del entrenamiento del modelo, la evolución por época de:

- **loss:** error del modelo en entrenamiento
- **accuracy:** precisión en entrenamiento
- **Otros parámetros**

Se utiliza para **analizar y graficar el desempeño del modelo** a lo largo del tiempo.

```
# Entrenar el modelo
historia = model.fit(
    x_train, y_train,      # Datos de entrenamiento
    epochs=5,             # Número de épocas
    batch_size=32,        # Tamaño del batch
    validation_split=0.1,  # Reservar el 10% para validación
    verbose=1              # Mostrar progreso detallado
)
```





Paso 4: Entrenar el modelo

Podemos Graficar la historia de entrenamiento del modelo para poder tener una idea de su evolución por épocas y tomar decisiones:



Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

```
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Entrenamiento MNIST - Red Neuronal Densa MLP", fontsize=14)

epocas_reales = range(1, len(historia.history["accuracy"]) + 1)
# Gráfica de Accuracy
ax1.plot(epocas_reales, historia.history["accuracy"], label="Train Accuracy", color="blue", linewidth=2)
ax1.plot(epocas_reales, historia.history["val_accuracy"], label="Val Accuracy", color="orange", linewidth=2, linestyle="--")
ax1.set_title("Accuracy por Época")
ax1.set_xlabel("Época")
ax1.set_ylabel("Accuracy")
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_ylim([0.8, 1.01])

# Gráfica de Pérdida (Loss)
ax2.plot(epocas_reales, historia.history["loss"], label="Train Loss", color="blue", linewidth=2)
ax2.plot(epocas_reales, historia.history["val_loss"], label="Val Loss", color="orange", linewidth=2, linestyle="--")
ax2.set_title("Pérdida (Loss) por Época")
ax2.set_xlabel("Época")
ax2.set_ylabel("Loss")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
#ruta_grafica = os.path.join(DATA_DIR, "TLP_OPTIMO-curvas_entrenamiento.png")
#plt.savefig(ruta_grafica, dpi=150)
#print(f"[INFO] Gráfica guardada en: {ruta_grafica}")
plt.show()
```





Paso 5: Evaluar el modelo

Después de entrenar, necesitamos saber qué tan bien funciona nuestro modelo en datos que **no ha visto antes**, para medir su capacidad de generalización.

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Método `model.evaluate()`

- Calcula la función de pérdida y las métricas definidas (como accuracy) en el **conjunto de prueba** (`x_test`, `y_test`).
- Retorna valores numéricos que resumen el desempeño del modelo.

Interpretación

- **Pérdida baja** indica que las predicciones están cerca de los valores reales.
- **Alta precisión** significa que el modelo clasifica correctamente la mayoría de las imágenes.

Importancia de esta etapa

Evaluar en datos nuevos es esencial para detectar:

- **Overfitting**: si el rendimiento en prueba es mucho peor que en entrenamiento.
- **Underfitting**: si el modelo no aprende lo suficiente y falla en ambos conjuntos.

```
test_loss, test_accuracy = model.evaluate(x_test,y_test,verbose=0)
```

```
print("=====")
print("===== TEST REPORT =====")
print("=====")

print(f"{'Muestras de Test':<25}: {len(x_test):,}")
print(f"{'Loss':<25}: {test_loss:.6f}")
print(f"{'Accuracy':<25}: {test_accuracy:.6f}")
print(f"{'Accuracy (%)':<25}: {test_accuracy * 100:.2f}%")
print(f"{'Error (%)':<25}: {(1 - test_accuracy) * 100:.2f}%")

if test_accuracy >= 0.99:
    estado = "EXCELENTE"
elif test_accuracy >= 0.98:
    estado = "MUY BUENO"
elif test_accuracy >= 0.95:
    estado = "BUENO"
else:
    estado = "MEJORABLE"

print(f"{'Clasificación':<25}: {estado}")

print("=====")
```



Paso 6: Guardar el modelo

Una vez que el modelo está entrenado y evaluado, es buena práctica **guardarlo** para:

- Reutilizarlo más adelante sin volver a entrenar.
- Compartirlo o desplegarlo en producción.
- Cargarlo para seguir entrenando más adelante (fine-tuning).

Opciones para guardar el modelo

Keras permite guardar el modelo **completo** (estructura + pesos + configuración de entrenamiento) en un solo archivo, usualmente con extensión .keras.

Este archivo incluye:

- La arquitectura del modelo
- Los pesos entrenados
- La función de pérdida, el optimizador y métricas
- El estado del entrenamiento (si se continúa luego)

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

```
print("=====")
print("===== MODELO GUARDADO =====")
print("=====")
print(f"[INFO] Modelo guardado en: {full_path_model}")
print(f"[INFO] Historia guardada en: {full_path_historia}")
print("=====")
```



```
# =====
# GUARDADO (MODELO E HISTORIA)
# =====
MODEL_DIR = r"C:\000-LABORATORIO_IA\00-ESCRITORIO\TP1\models"
DATA_DIR = r"C:\000-LABORATORIO_IA\00-ESCRITORIO\TP1\data"

os.makedirs(MODEL_DIR, exist_ok=True) # Crear carpetas si no existen
os.makedirs(DATA_DIR, exist_ok=True) # Crear carpetas si no existen

# 1. Definir nombres de archivo
timestamp = datetime.now().strftime("%Y.%m.%d-%H.%M.%S")
model_name = f"{timestamp}-Modelo_Mnist.keras"
historia_name = model_name[:-6] + "-historia.npy"

# 2. Guardar la Modelo
full_path_model = os.path.join(MODEL_DIR, model_name)
model.save(full_path_model)

# 3. Guardar la Historia
full_path_historia = os.path.join(DATA_DIR, historia_name)

historia_dict = {metrica: [float(v) for v in valores] for metrica,
valores in historia.history.items()}

np.save(full_path_historia, historia_dict)
```



SEPARAR EL SCRIPT PARA MEJOR VISIBILIDAD.

Separa el Script en 2: Uno de **ENTRENAMIENTO_CORE.py** y otro **REPORTING_UTIL.py** para tener mejor visibilidad del proceso:

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

ENTRENAMIENTO_CORE.py

```
# =====
# MODELO
# =====
model = Sequential([])
model.compile()

print_model_config(model)

# =====
# DATASET
# =====
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print_dataset_info(x_train, y_train, x_test, y_test)

# =====
# TRAINING
# =====
history = model.fit()

plot_training(history)

# =====
# EVALUATION
# =====
test_loss, test_acc = model.evaluate()

print_test_report(test_loss, test_accuracy, x_test)
```

REPORTING_UTIL.py

```
# =====
# MODELO
# =====
def print_model_config(model):
    model.summary()

# =====
# DATASET
# =====
def print_dataset_info(x_train, y_train, x_test, y_test):

# =====
# TRAINING
# =====
def plot_training(history):
    plt.show()

# =====
# EVALUATION
# =====
def print_test_report(test_loss, test_accuracy, x_test):
```

Tengo este script de entrenamiento de redes neuronales, separamelo en dos scripts uno que contenga todas las líneas de entrenamiento y uno secundario que contenga funciones de presentación en stdout, y graficado.



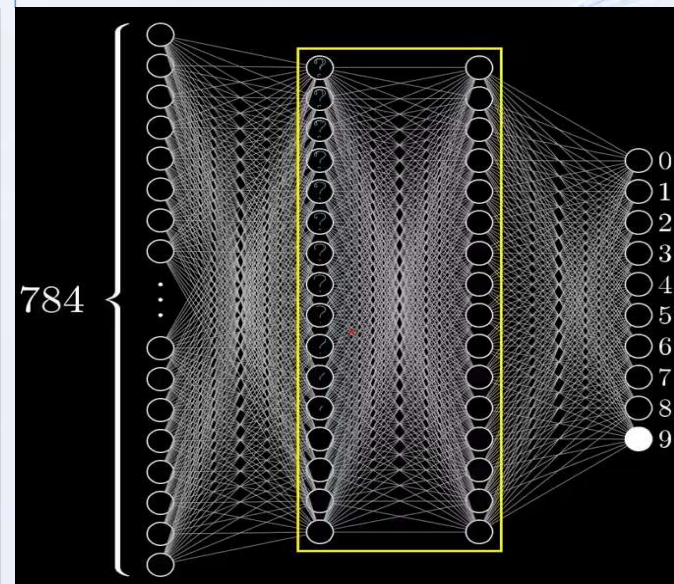
Paso 7: Ajustar o mejorar el modelo

Una vez evaluado el rendimiento, podemos **refinar** el modelo para obtener mejores resultados. Este paso es **iterativo**: probamos, evaluamos y volvemos a ajustar. Se conoce como **ajuste de hiperparámetros** o **tuning**:

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Estrategia	Descripción breve
Cambiar la arquitectura	Más capas ocultas o más neuronas por capa pueden capturar más complejidad.
Cambiar la función de activación	Por ejemplo, usar leaky_relu, tanh, etc., según el tipo de datos.
Regularización	Añadir capas como Dropout o BatchNormalization para evitar sobreajuste.
Más épocas	Entrenar durante más tiempo puede mejorar el aprendizaje (pero ojo con el overfitting).
Cambiar el optimizador	Probar SGD, RMSprop, Nadam, etc., puede influir en la velocidad de convergencia.
Modificar la tasa de aprendizaje	Ajustar este hiperparámetro puede mejorar estabilidad y precisión.
Aumentar los datos (data augmentation)	Si el dataset es pequeño, crear variaciones sintéticas mejora la generalización.
Normalizar o estandarizar mejor los datos	Escalar correctamente puede facilitar el aprendizaje.

MODELO PROPUESTO





Paso 7: Ajustar o mejorar el modelo

[AGREGAR BATCH NORMALIZATION]

Etapa		Aspecto	Descripción
1	Definir el modelo	¿Qué es?	Técnica que normaliza las activaciones de una capa
2	Compilar el modelo	Objetivo	Estabilizar y acelerar el entrenamiento
3	Preparar los datos	Cómo funciona	Normaliza usando media y varianza del batch
		Parámetros entrenables	gamma (escala), beta (shift)
4	Entrenar el modelo	Estadísticas internas	moving_mean, moving_variance
		Fórmula	$y = \gamma * (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$
		En entrenamiento	Usa estadísticas del batch actual
5	Evaluar el modelo	En inferencia	Usa medias móviles acumuladas
		Efecto en entrenamiento	Más rápido y estable
6	Guardar el modelo	Efecto en loss	Reduce oscilaciones del loss
		Riesgo	Dependencia del batch size (muy pequeño = ruido)
7	Ajustar o mejorar	Intuición	“Mantener todas las capas hablando el mismo idioma numérico”
8	Cargar y usar el modelo	Impacto en MNIST	Generalmente mejora accuracy y velocidad de convergencia

```

model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),

    Dense(700, kernel_initializer=HeNormal()),
    BatchNormalization(),
    Activation("relu"),
    Dropout(0.3),

    Dense(500, kernel_initializer=HeNormal()),
    BatchNormalization(),
    Activation("relu"),
    Dropout(0.3),

    Dense(10, activation="softmax")
])
    
```





Paso 7: Ajustar o mejorar el modelo

[AGREGAR BATCH NORMALIZATION]

BatchNorm hace que cada capa reciba datos “ordenados y consistentes”, evitando que la red tenga que adaptarse a valores que cambian constantemente durante el entrenamiento. En otras palabras, durante el aprendizaje, la red no trabaja con valores caóticos o escalas muy distintas, sino con señales estabilizadas.

Contrapartida en inferencia

La contrapartida es que el modelo **ya no depende solo de los pesos (W) y sesgos (b)**. Ahora también necesita información adicional que aprendió durante el entrenamiento:

- media acumulada de cada activación
- varianza acumulada de cada activación

Es decir, el modelo ya no es simplemente:

$$y = W \cdot x + b$$

sino que internamente incluye una normalización:

$$y = \gamma \cdot (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

Implicancia importante

Esto significa que en inferencia el modelo:

- no puede funcionar solo con pesos y biases
- necesita también las estadísticas aprendidas (μ y σ^2)
- depende de una “memoria estadística” del entrenamiento

Conclusión final:

Sin BatchNorm, el modelo es como una máquina mecánica simple:

- **Solo necesita engranajes (W y b)**

Con BatchNorm, es como una máquina que además tiene un sistema de calibración interno:

- *** Necesita engranajes + sensores de referencia (media y varianza aprendidas)**

Esto es lo que hace que sea más estable y eficiente, pero también un poco más complejo de portar o implementar manualmente en otros entornos.



```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),

    Dense(700, kernel_initializer=HeNormal()),
    BatchNormalization(),
    Activation("relu"),
    Dropout(0.3),

    Dense(500, kernel_initializer=HeNormal()),
    BatchNormalization(),
    Activation("relu"),
    Dropout(0.3),

    Dense(10, activation="softmax")
])
```



Paso 7: Ajustar o mejorar el modelo

[AGREGAR BATCH NORMALIZATION]

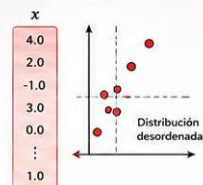
BATCH NORMALIZATION: CÓMO FUNCIONA

NORMALIZA, ESTABILIZA Y ACELERA EL APRENDIZAJE



FLUJO EN ENTRENAMIENTO (POR CADA BATCH)

1. ENTRADA ACTIVACIONES SIN NORMALIZAR



2. CALCULAR ESTADÍSTICAS DEL BATCH

Media (μ)

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i$$

Varianza (σ^2)

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$$

Se calculan la media y la varianza de las activaciones del batch.

3. NORMALIZAR (CENTRAR Y ESCALAR)

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

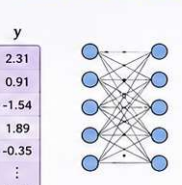
Las activaciones quedan con media ~ 0 y varianza ~ 1 .

4. ESCALAR Y DESPLAZAR (PARÁMETROS APRENDIBLES)

$$y_i = \gamma \hat{x}_i + \beta$$

γ (gamma) escala y β (beta) desplaza para que la red recupere la capacidad de representación.

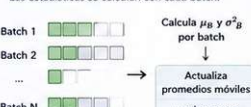
5. SALIDA NORMALIZADA PARA LA SIGUIENTE CAPA



ENTRENAMIENTO vs INFERENCIA

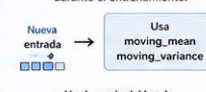
DURANTE ENTRENAMIENTO

Las estadísticas se calculan con cada batch.



DURANTE INFERENCIA (PREDICCIÓN)

Se usan las estadísticas acumuladas durante el entrenamiento.



¿QUÉ APORTA AL MODELO?



PARÁMETROS DE BATCH NORMALIZATION

Parámetros aprendibles

γ (gamma) Escala

β (beta) Desplazamiento

Se aprenden con backpropagation.

Estadísticas no aprendibles

μ_{moving_mean} (μ) Media acumulada

$\sigma_{moving_variance}^2$ (σ^2) Varianza acumulada

Se actualizan en cada batch con un factor de momentum (ej. 0.99).

ϵ (epsilon)

Número muy pequeño (ej. $1e-5$)

Evita división por cero y da estabilidad numérica.

¿QUÉ NECESITA EL MODELO PARA INFERENCIA?

Un modelo con BatchNorm NO es solo W y B. Necesita también:

Pesos (W) de las capas

Sesgos (B) de las capas

γ y β (por cada BatchNorm)

μ_{moving_mean} (por cada BatchNorm)

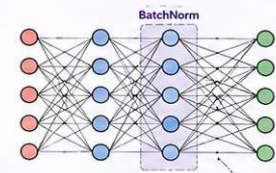
$\sigma_{moving_variance}^2$ (por cada BatchNorm)

ϵ (epsilon) (por cada BatchNorm)

⚠ Sin estas estadísticas, las predicciones serán incorrectas.

¿QUÉ ES?

Batch Normalization (BatchNorm) es una técnica que normaliza las activaciones de una capa para que tengan media ~ 0 y varianza ~ 1 durante el entrenamiento.



¿PARA QUÉ SIRVE?

- ✓ Estabiliza las distribuciones entre capas
- ✓ Acelera la convergencia
- ✓ Permite usar learning rates más altos
- ✓ Reduce la sensibilidad a la inicialización
- ✓ Actúa como regularizador ligero

IDEA INTUITIVA

Hace que cada capa reciba datos "ordenados y consistentes", evitando que la red tenga que adaptarse a valores que cambian constantemente.



FÓRMULA COMPLETA EN INFERENCIA

Dado un valor x :

$$y = \gamma \cdot \frac{x - \mu_{moving_mean}}{\sqrt{\sigma_{moving_variance}^2 + \epsilon}} + \beta$$

No usa el batch actual, usa estadísticas acumuladas.

IDEA CLAVE



Inicialización mejor



Señales estables entre capas



Gradientes más saludables



Entrenamiento más rápido y eficiente



Mejor generalización en datos nuevos



Paso 7: Ajustar o mejorar el modelo

[AGREGAR DROPOUT]

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Aspecto	Descripción
¿Qué es?	Técnica de regularización que apaga neuronas aleatoriamente durante el entrenamiento
Objetivo	Evitar overfitting (que la red memorice los datos)
Cómo funciona	En cada batch, desactiva aleatoriamente un % de neuronas
Parámetro principal	rate (ej: 0.3 = 30% de neuronas apagadas)
Fórmula conceptual	Durante entrenamiento: $y = x * \text{mask}$ donde $\text{mask} \in \{0,1\}$
En inferencia	NO se aplica dropout (todas las neuronas activas)
Efecto en entrenamiento	Aumenta dificultad → loss suele subir
Efecto en generalización	Mejora rendimiento en datos no vistos
Riesgo si es alto	Underfitting (modelo aprende poco)
Intuición	“Entrenar una red más robusta obligándola a no depender de neuronas específicas”
Impacto en MNIST	Puede ayudar, pero valores altos (0.5) suelen ser excesivos

```

model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),

    Dense(700, kernel_initializer=HeNormal()),
    BatchNormalization(),
    Activation("relu"),
    Dropout(0.3),

    Dense(500, kernel_initializer=HeNormal()),
    BatchNormalization(),
    Activation("relu"),
    Dropout(0.3),

    Dense(10, activation="softmax")
])
    
```



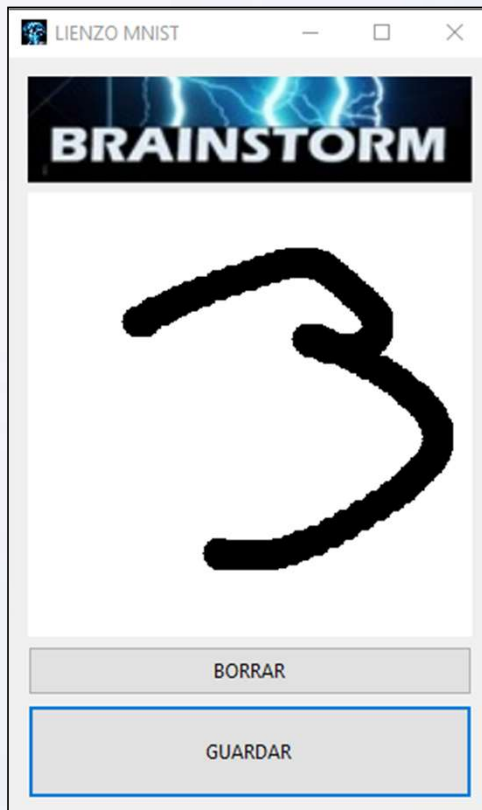


Paso 8: Cargar y usar el modelo

Etapa	
1	Definir el modelo
2	Compilar el modelo
3	Preparar los datos
4	Entrenar el modelo
5	Evaluar el modelo
6	Guardar el modelo
7	Ajustar o mejorar
8	Cargar y usar el modelo

Una vez que tenemos un modelo guardado en disco, podemos cargarlo para:

- Realizar predicciones en nuevos datos.
- Evaluar su rendimiento.
- Continuar entrenándolo.
- load_model**
Función de Keras que permite **cargar un modelo previamente guardado** (archivo .h5 o carpeta SavedModel). Recupera toda la información del modelo: arquitectura, pesos, configuración y estado. Así, podés usar el modelo sin necesidad de definirlo o entrenarlo de nuevo.
- predict**
Método del modelo cargado que recibe nuevos datos de entrada y devuelve **las predicciones del modelo**. Para clasificación, suele entregar probabilidades por clase, que luego pueden interpretarse para obtener la clase más probable.



```
import numpy as np
from tensorflow.keras.models import load_model
#####
# Cargar el modelo guardado
modelo_cargado = load_model("C:\\000-LABORATORIO_IA\\mi_modelo.h5")
#####
# CARGAR EL TXT COMO IMAGEN
with open("C:\\000-LABORATORIO_IA\\img.txt", "r") as f:
    valores = [float(line.strip()) for line in f.readlines()]

# Convertir a matriz 28x28
img_input = np.array(valores).reshape(1, 28, 28) # sin canal
#####
# Predecir
prediccion = modelo_cargado.predict(img_input)
clase = np.argmax(prediccion)

print(f"Dígito predicho: {clase}")
#####
# Mostrar como imagen
import matplotlib.pyplot as plt
plt.imshow(img_input[0], cmap="gray")
plt.title("Imagen cargada desde img.txt")
plt.axis("off")
plt.show()
#####
```



Crea un dibujo con el programa LIENZO MNIST y prueba la capacidad de tu red neuronal!



CALLBACKs de TensorFlow

En TensorFlow, **los callbacks** son objetos que permiten **personalizar y controlar el proceso de entrenamiento** de un modelo. Actúan como "LLAMADAS" que se ejecutan automáticamente en momentos específicos durante el entrenamiento, evaluación o predicción de un modelo, como al inicio o fin de una época, o al detectar mejoras en alguna métrica.

Un callback en Keras/TensorFlow es una clase que hereda de `tf.keras.callbacks.Callback` y define funciones que se ejecutan **en distintos puntos del ciclo de entrenamiento**. Estas funciones se pueden utilizar para:

- Detener el entrenamiento si ya no mejora
- Guardar el modelo en puntos clave
- Registrar estadísticas personalizadas
- Cambiar dinámicamente el learning rate
- Mostrar resultados o graficar métricas en tiempo real

Las funciones dentro de un callback se ejecutan en los siguientes eventos:
Al **inicio o fin** de:

- una **época**
- un **batch**
- el **entrenamiento completo**
- Cuando se alcanza una **condición específica** (ej: mejora en `val_loss`)

```
BEST_MODEL_NAME = f"{timestamp}-MNIST-BEST.keras"
RUTA_MEJOR_MODELO = os.path.join(MODEL_DIR, BEST_MODEL_NAME)
```

```
from keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
```

```
# 1. EarlyStopping → evita overfitting
```

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=5,           # MNIST converge rápido → 3-7 ideal
    min_delta=1e-4,
    restore_best_weights=False,
    verbose=1
)
```



```
# 2. ModelCheckpoint → guarda el mejor modelo REAL
```

```
checkpoint = ModelCheckpoint(
    filepath=RUTA_MEJOR_MODELO,
    monitor="val_loss",
    save_best_only=True,
    save_weights_only=False,
    verbose=1
)
```

```
# 3. ReduceLROnPlateau → afina convergencia
```

```
reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=2,
    min_lr=1e-5,
    verbose=1
)
```

```
callbacks = [early_stop, checkpoint, reduce_lr]
```



CALLBACKS de TensorFlow

Objetivo	Callback recomendado
Guardar el mejor modelo	ModelCheckpoint
Evitar entrenar de más	EarlyStopping
Afinar el learning rate automáticamente	ReduceLROnPlateau
Visualizar métricas	TensorBoard, CSVLogger
Automatizar lógica personalizada	LambdaCallback, LearningRateScheduler
Reanudar después de interrupciones	BackupAndRestore

#En este ejemplo:



```
callbacks = [
    tf.keras.callbacks.ModelCheckpoint('mejor_modelo.h5',
    save_best_only=True),
    tf.keras.callbacks.EarlyStopping(monitor='val_loss',
    patience=5),
    tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss',
    factor=0.5, patience=3),
    tf.keras.callbacks.CSVLogger('entrenamiento.csv')
]
```

```
model.fit(X_train, y_train, epochs=100,
validation_data=(X_val, y_val), callbacks=callbacks)
```

Callback	Función principal	Parámetros clave	Cuándo se usa
ModelCheckpoint	Guarda automáticamente el modelo durante el entrenamiento	filepath, save_best_only, monitor, mode	Cuando quieres conservar el mejor modelo mientras entrena
EarlyStopping	Detiene el entrenamiento si una métrica deja de mejorar	monitor, patience, min_delta, restore_best_weights	Para evitar overfitting o entrenamientos innecesariamente largos
ReduceLROnPlateau	Reduce el learning rate cuando una métrica se estanca	monitor, factor, patience, min_lr	Cuando el entrenamiento se frena y quieres afinar el aprendizaje
TensorBoard	Registra datos para visualizarlos en TensorBoard	log_dir, histogram_freq, write_graph	Para monitorear métricas, arquitectura, imágenes, etc.
CSVLogger	Guarda métricas por época en un archivo .csv	filename, separator, append	Cuando quieres registrar los resultados para análisis posterior
TerminateOnNaN	Detiene el entrenamiento si aparece un NaN en pérdida o métricas	-	Para prevenir que un error silencioso dañe el entrenamiento
LearningRateScheduler	Ajusta dinámicamente el learning rate con una función definida por el usuario	schedule, verbose	Cuando quieres controlar el LR según la época o una condición
ProgbarLogger	Muestra una barra de progreso por época y batch	Automático (activado por defecto)	Para información visual del entrenamiento en consola
BackupAndRestore	Guarda el progreso del entrenamiento y lo restaura si se interrumpe	backup_dir	Entrenamientos largos o distribuidos donde puede haber cortes
RemoteMonitor	Envía métricas a un servidor HTTP para monitoreo externo	root, path, field, headers	Entrenamiento remoto, dashboards personalizados
LambdaCallback	Permite ejecutar funciones personalizadas en distintos puntos del entrenamiento	on_epoch_end, on_batch_begin, etc.	Para agregar lógica personalizada sin crear una clase completa
BaseLogger	Registra las métricas estándar por época	Interno (no suele usarse directamente)	Complementa a otros callbacks
History	Guarda automáticamente el historial del entrenamiento	Automático (devuelto por model.fit())	Acceso a métricas post-entrenamiento



Manejo de Modelos Pre-Entrenados:

Esa sección del programa es la encargada de **inicializar el modelo de red neuronal**, pero con una lógica que permite **aprovechar un entrenamiento previo si está disponible**.

La idea es simple:

Si ya existe un archivo .h5 en la ruta H5_PATH (que contiene un modelo previamente entrenado y guardado), el código lo **carga directamente** con `tf.keras.models.load_model()`. Esto evita tener que volver a entrenar desde cero, conservando tanto la arquitectura como los pesos aprendidos.

Si no existe ese archivo, el programa **crea un nuevo modelo** usando `Sequential`, definiendo explícitamente las capas:

- Una **capa de entrada** que recibe 40 características (`shape=(40,)`).
- Dos **capas ocultas** de 10 neuronas cada una, con activación sigmoid e inicialización GlorotUniform (ideal para este tipo de funciones de activación).
- Una **capa de salida** con 8 neuronas (correspondientes a las 8 salidas del problema), también con sigmoid para producir probabilidades independientes por salida.

En resumen, esta parte del código implementa un **sistema híbrido de inicialización**:

- **Modo reutilización** → Si ya hay un modelo guardado, se carga y continúa entrenando desde donde quedó.
- **Modo inicialización** → Si no existe modelo previo, se arranca desde cero con pesos inicializados aleatoriamente según la estrategia elegida.

```
##### CARGO O CREO EL MODELO #####
if os.path.exists(H5_PATH):
    print(f"Cargando modelo preentrenado desde {H5_PATH}...")
    model = tf.keras.models.load_model(H5_PATH)
else:
    print("No se encontró modelo preentrenado. Creando uno nuevo...")
    model = tf.keras.models.Sequential([
        tf.keras.layers.Input(shape=(40,)),
        tf.keras.layers.Dense(10, activation='sigmoid',
                                kernel_initializer=tf.keras.initializers.GlorotUniform()),
        tf.keras.layers.Dense(10, activation='sigmoid',
                                kernel_initializer=tf.keras.initializers.GlorotUniform()),
        tf.keras.layers.Dense(8, activation='sigmoid',
                                kernel_initializer=tf.keras.initializers.GlorotUniform())
    ])

#####
```





```
01 Sapest openai
02 from langchain import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10     respuesta = modelo.invoke(
11     ) prompt
12     return respuesta.content
13 )
14
15 if __name__ == "__main__":
16     print("Gracias !")
17     print("dudas, consultas...")
18     |
```

> print("Gracias !")

> print("dudas, consultas...")

> input("Presione una tecla para salir...")



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383

